

Relatório AED- PROJETO 2

TAD Graph

Dinis Oliveira, 119193

André Silva, 119480

Turma P9

Considerações Gerais

Este relatório aborda a implementação e análise do algoritmo Bellman-Ford, usado para encontrar caminhos mínimos em grafos orientados sem pesos associados às arestas, incluindo a detecção de ciclos negativos. O documento também explora sua aplicação na construção do fecho transitivo de um grafo orientado sem pesos, utilizando o módulo Bellman-Ford para determinar os vértices alcançáveis a partir de um dado vértice.

Algoritmo de Bellman-Ford

O algoritmo Bellman-Ford está destinado para encontrar o menor caminho de um vértice inicial para todos os outros vértices de um grafo.

O algoritmo recebe como argumentos um grafo e um vértice inicial e retorna um grafo com os caminhos mais curtos a partir do vértice inicial

Vértice: V ; Aresta: E ; Caminho máximo entre 2 vértices: l

Descrição do Algoritmo e Análise de Complexidade Temporal

1. Inicialização

Para cada vértice, inicializar os valores de distância ('Infinito'), vértice antecessor (a um inexistente, -1) e marcar como não atingido (0). O vértice

Isto depende, apenas, do número de vértices, logo:

Complexidade: $O(V)$

2. Procura do caminho mais curto

Relaxamento de cada aresta: verificar caminho mais curto em vértice adjacente.

A iteração ocorre $V-1$ vezes, que corresponde ao caminho mais longo possível num grafo. E em cada uma dessas iterações, itera-se sobre cada aresta E (verificar existência de caminho mais curto via vértice adjacente).

Logo, isto depende do número de vértices e arestas:

Complexidade: $O(V \cdot E)$

Melhoria da 2ª fase do Algoritmo

Adicionando, uma ‘flag’ para verificar se em alguma iteração, do ciclo externo, não ocorre nenhum relaxamento de um vértice, permite interromper o ciclo, pois não há mais possibilidade de processar arestas, e encontrar um caminho mais curto.

Isto permite melhoria na complexidade, pois o ciclo externo deixa de depender de $V-1$ e passa a depender comprimento do caminho mais longo entre o vértice inicial e qualquer outro vértice alcançável (que em muitos grafos é menos de metade de V).

Logo,

Complexidade: $O(l \cdot E)$

3. Detecção de ciclos negativos

A verificação de presença de ciclos negativos, ocorre iterando sobre as arestas do grafo.

Logo,

Complexidade: $O(E)$

Complexidade Total de Tempo

A complexidade Total resulta da soma das complexidades das 3 fases, logo:

$$\begin{aligned} &O(V) + O(l \cdot E) + O(E) \\ &= O(V + l \cdot E + E) = O(V + l \cdot E) \approx O(V \cdot E) \end{aligned}$$

Análise da Complexidade de espaço ocupado em memória

A Estrutura de Dados do Módulo do Algoritmo Bellman-Ford, é composta por:

- 1 Grafo, a representação do grafo cresce consoante o número de vértices e arestas: $O(V \cdot E)$
- 3 arrays de inteiros, que crescem com o número de vértices: $O(V)$
- 1 índice de um vértice (inteiro): $O(1)$
- Lista de adjacência temporária, que em média crescem com E : $O(E)$

Logo,

$$O(V \cdot E) + 3O(V) = O(V \cdot E)$$

Dados experimentais

Foram realizados testes do tempo em milissegundos de execução do algoritmo. Foram usados os grafos já fornecidos nos testes dados, dig01, g01 e DG_2. E os restantes foram os fornecidos em ficheiros de texto.

Foram analisados tempos para o algoritmo com e sem a flag para quebra do ciclo da 2ª fase do algoritmo. Notando-se apenas ligeiras melhorias (Grafos pequenos).

Grafo	Nº. Vértices	Nº. Arestas	Tempo(ms)	Tempo(ms) - sem flag
dig01	6	3	0.066227	0.070946
g01	6	8	0.111818	0.119426
DG_1	6	6	0.133000	0.141891
DAG_1	7	9	0.155450	0.159892
DAG_2	7	12	0.158539	0.163666
DAG_3	7	11	0.158539	0.155610
DAG_4	15	25	0.530051	0.526844
DG_2	15	26	0.613254	0.685094

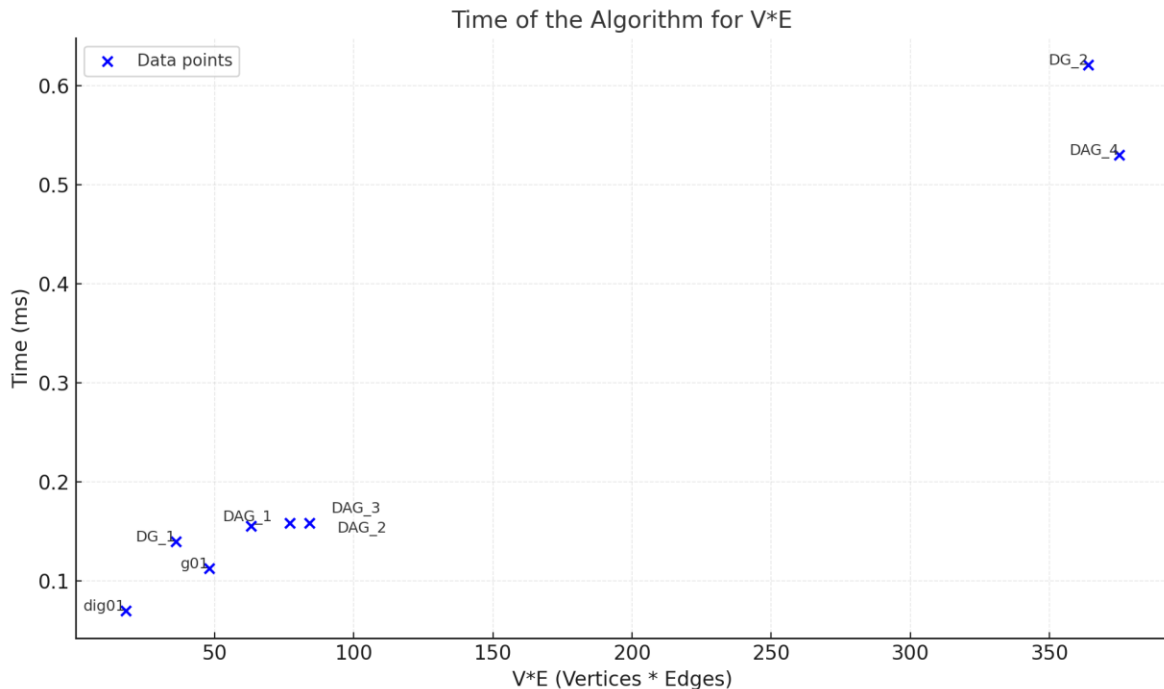


Tabela 1 e gráfico 1 – Testes temporais para o Algoritmo Bellman-Ford (Gráfico apenas para o Algoritmo melhorado)

Esta análise experimental, permite revalidar que o tempo de execução do algoritmo depende do número de Arestas e Vértices de um dado grafo.

Algoritmo de Construção do Fecho Transitivo

Descrição do Algoritmo

1. Inicialização

Inicializa-se o algoritmo com o número de vértices do grafo original em $O(1)$ e cria-se um grafo vazio para armazenar o fecho transitivo. A criação do grafo requer $O(V + E)$, já que é necessário alocar memória para armazenar todos os vértices e arestas.

2. Iteração sobre os vértices;

O algoritmo percorre todos os vértices do grafo original, realizando uma chamada ao algoritmo de Bellman-Ford para cada um deles. Esse processo envolve V iterações externas, cada uma executando Bellman-Ford em tempo $O(V.E)$, resultando em uma complexidade total de $O(V(V.E))$.

3. Adição das arestas de acordo com o algoritmo Bellman-Ford;

Dentro de cada execução de Bellman-Ford, o algoritmo verifica a acessibilidade entre os pares de vértices e adiciona uma aresta no grafo do fecho transitivo.

Complexidade Total do Tempo

A complexidade Total do tempo vai tomar em conta a complexidade do algoritmo sem contar com a inicialização do grafo, logo:

$$\begin{aligned} O(V(V.E)) &= \\ &= O(V^2.E) \end{aligned}$$

Análise Do Espaço Ocupado Em Memória

A estrutura de dados utilizada no algoritmo de Fecho Transitivo é composta por:

- 1 Grafo para o fecho transitivo, cuja representação cresce com o número de vértices e arestas: $O(V + E)$

- 1 Algoritmo Bellman-Ford executado para cada vértice, que como vimos antes tem: $O(V(V.E))$

Logo,
 $O(V + E + V^2.E)$

Dados Experimentais

Realizamos novamente testes do tempo em milissegundos de execução do algoritmo. Foram usados os grafos já fornecidos nos testes dados, dig01, g01 e DG_2. E os restantes foram os fornecidos em ficheiros de texto.

Grafo	Nº. Vértices	Nº. Arestas	Tempo(ms)
dig01	6	3	0.057358
DG_1	6	6	0.466996
DAG_1	7	9	0.476874
DAG_2	7	12	0.483977
DAG_3	7	11	0.454181
DAG_4	15	25	0.490299
DG_2	15	26	0.536004

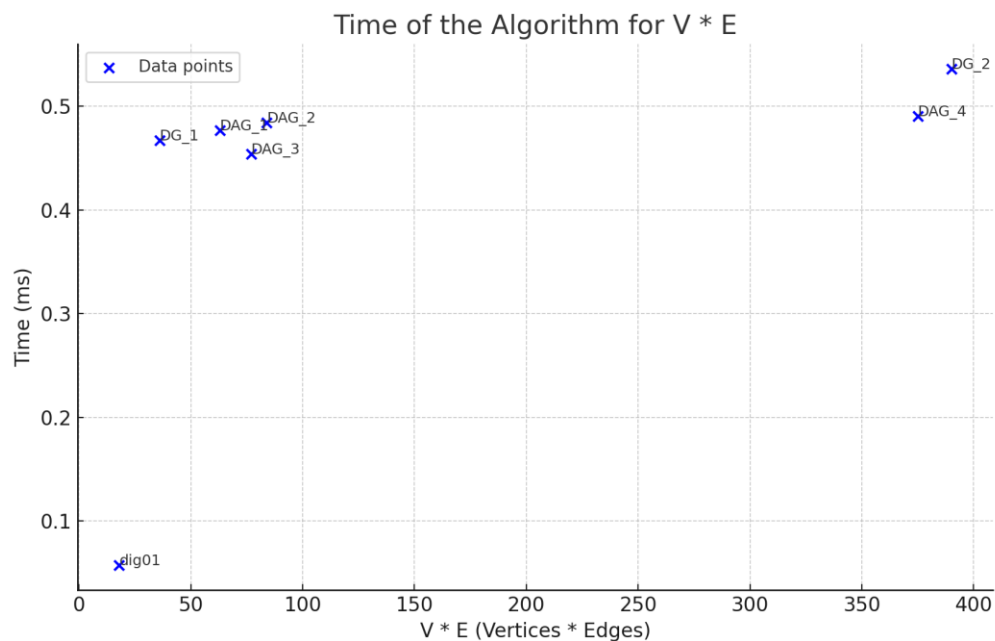


Tabela 2 e gráfico 2 – Testes temporais para o Algoritmo de Construção do Fecho Transitivo

Analisando estes dados experimentais, percebemos mais uma vez que o tempo de execução do algoritmo depende do número de Arestas de Vértices e um dado grafo.

Conclusão Final

Este trabalho proporcionou o aperfeiçoamento de competências na implementação e análise de algoritmos para grafos orientados sem pesos e na sua construção. Desenvolvemos o grafo transposto, calculamos os caminhos mais curtos entre vetores através do algoritmo de Bellman-Ford, criamos um grafo de fecho transitivo, e por fim as distâncias entre todos os pares de vértices e medidas de excentricidade. Através de uma metodologia de testes com grafos simples e complexos, e análises de complexidade, validamos a eficiência e o bom desempenho dos algoritmos implementados.